REPORT

Zajęcia: Analog and digital electronic circuits
Teacher: prof. dr hab. Vasyl Martsenyuk

Lab 5
Date 27.03.26
Topic: "Random Signal Analysis. Power Spectrum Estimation"
Variant 13

Kostiantyn Hrekov
Informatyka II stopień,
stacjonarne,
1 semestr,
Gr 1.a

1. Problem statement:

Variant 13.

Simulate two random signals with different power levels. Analyze and compare their energy and standard deviation. Estimate and compare their PSDs using Welch with Hamming and Hann windows.

Input data:

N = 5000 # number of samples
fs = 1000 # sampling frequency (Hz)
signal1 = np.random.normal(0, 1, N) # lower power
signal2 = np.random.normal(0, 3, N) # higher power

2. Commands used (or GUI):

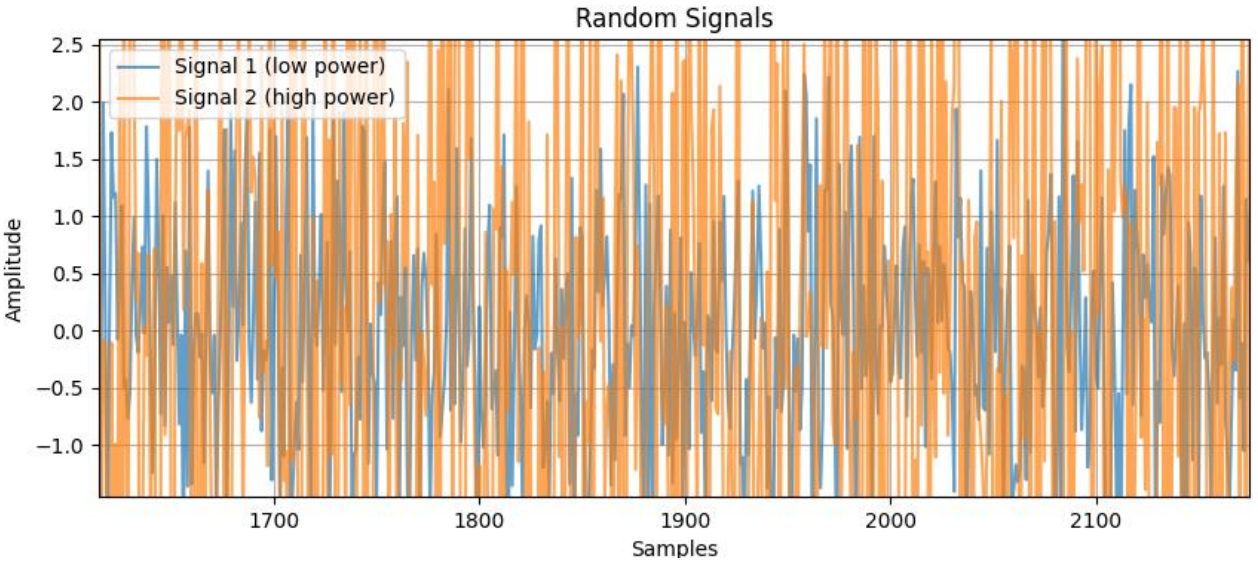
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import welch

# ========================
# 1. Parameters
# ========================
N = 5000          # number of samples
fs = 1000         # sampling frequency (Hz)

# ========================
# 2. Generate signals
# ========================
# Two random signals with different power levels
signal1 = np.random.normal(0, 1, N)   # lower power
signal2 = np.random.normal(0, 3, N)   # higher power

# ========================
# 3. Plot signals
# ========================
plt.figure(figsize=(10,4))

plt.plot(signal1, label='Signal 1 (low power)', alpha=0.7)
plt.plot(signal2, label='Signal 2 (high power)', alpha=0.7)

plt.title("Random Signals")
plt.xlabel("Samples")
plt.ylabel("Amplitude")
plt.legend()
plt.grid()

plt.show()

# ========================
# 4. Energy and standard deviation
# ========================
# Energy calculation
E1 = np.sum(signal1**2)
E2 = np.sum(signal2**2)

# Standard deviation
std1 = np.std(signal1)
std2 = np.std(signal2)
```

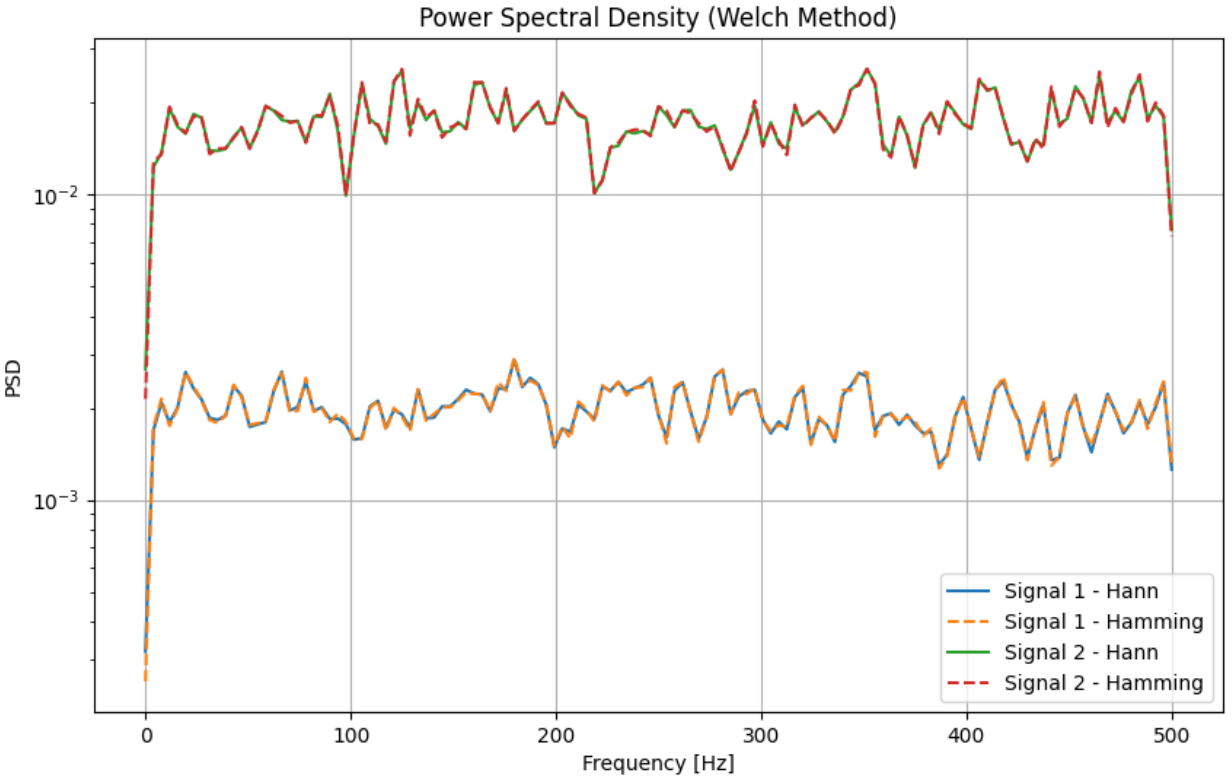
```python
print("=== RESULTS ===")
print(f"Signal 1 -> Energy: {E1:.2f}, STD: {std1:.2f}")
print(f"Signal 2 -> Energy: {E2:.2f}, STD: {std2:.2f}")

# ========================
# 5. PSD estimation (Welch method)
# ========================
# Using Hann window
f, Pxx1_hann = welch(signal1, fs=fs, window='hann')
f, Pxx2_hann = welch(signal2, fs=fs, window='hann')

# Using Hamming window
f, Pxx1_hamming = welch(signal1, fs=fs, window='hamming')
f, Pxx2_hamming = welch(signal2, fs=fs, window='hamming')

# ========================
# 6. Plot PSD
# ========================
plt.figure(figsize=(10,6))

# Signal 1 PSD
plt.semilogy(f, Pxx1_hann, label='Signal 1 - Hann')
plt.semilogy(f, Pxx1_hamming, '--', label='Signal 1 - Hamming')

# Signal 2 PSD
plt.semilogy(f, Pxx2_hann, label='Signal 2 - Hann')
plt.semilogy(f, Pxx2_hamming, '--', label='Signal 2 - Hamming')

plt.title("Power Spectral Density (Welch Method)")
plt.xlabel("Frequency [Hz]")
plt.ylabel("PSD")
plt.legend()
plt.grid()

plt.show()
```

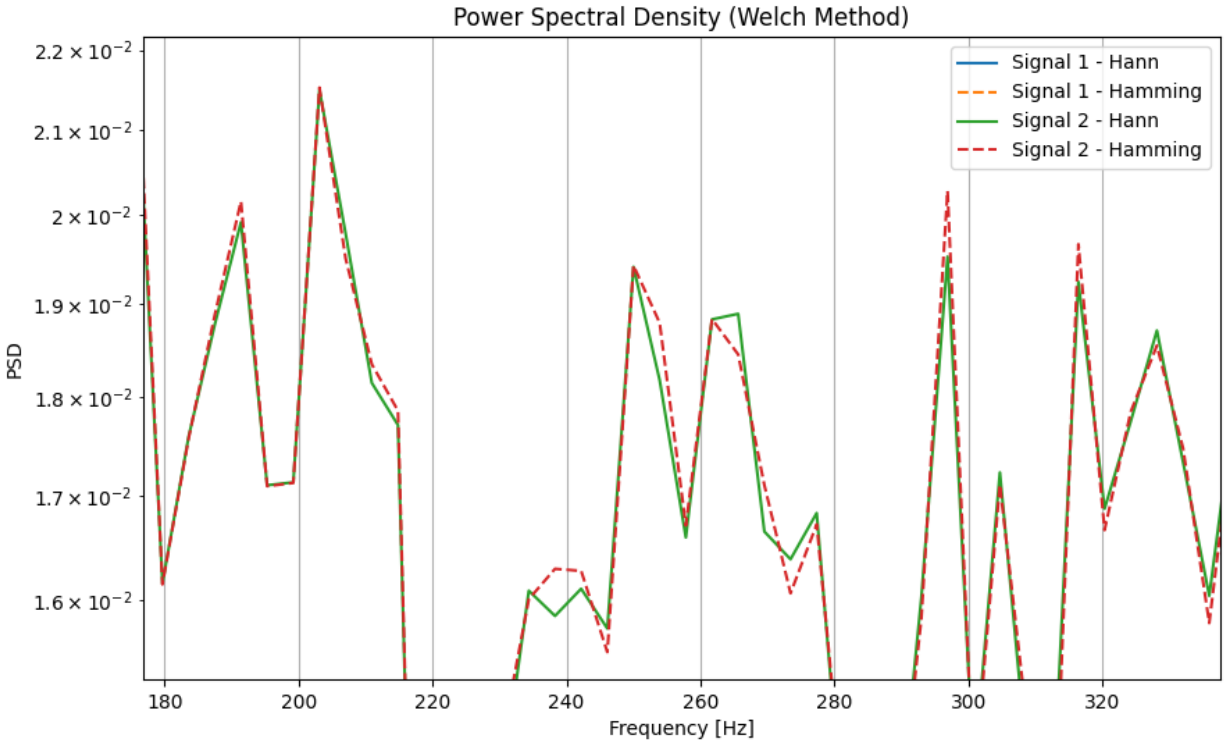
Output:

Signal 1 -> Energy: 4984.12, STD: 1.00
Signal 2 -> Energy: 45437.88, STD: 3.01

Signal 2 has significantly higher energy and standard deviation than Signal 1, confirming its higher power level. The results are consistent with the signal generation, where a larger variance leads to greater energy and amplitude dispersion.


Random Signals
Signal 1 (low power)
Signal 2 (high power)
Amplitude
Samples
2.5
2.0
1.5
1.0
0.5
0.0
−0.5
−1.0
1700
1800
1900
2000
2100


Power Spectral Density (Welch Method)
PSD
Frequency [Hz]
10^{-2}
10^{-3}
0
100
200
300
400
500
Signal 1 - Hann
Signal 1 - Hamming
Signal 2 - Hann
Signal 2 - Hamming



Conclusion:

Due to this lab, we learned how the power level of random signals affects their statistical properties and spectral representation. We observed that a signal with higher variance has significantly greater energy and standard deviation, while maintaining a similar spectral shape. We also saw that the choice of window in Welch′s method slightly influences the PSD estimate, highlighting the trade-off between spectral smoothing and resolution in practical analysis.

Link to remote repozytorium (e.g. GitHub):

https://github.com/KostiaGrekov/DSPja/tree/main/DSPja%205